# Java Programming

CPT111 – Lecture 10

Erick Purwanto

Xi'an Jiaotong-Liverpool University
西交利物浦大学

CPT111 Java Programming

Lecture 10

# File I/O and GUI with JavaFX

# Welcome!

- Welcome to Lecture 10 !
- Last week, we have learned about  OOP Principles and Exception
  - encapsulation, inheritance, and polymorphism
  - throwing and handling/catching exception
- In this lecture we are going to learn about
  - File I/O
    - learning Java File I/O mechanism
    - writing into and reading from a file
  - GUI with JavaFX
    - drawing shapes and creating graphical user interface
    - practicing OOP with JavaFX

# Learning Outcomes and Optional Textbook Reading

- Relevant Learning Outcomes:
  C. Be able to develop programs that the user can interact with either through graphical representation, or text based menus;
  D. Use Java programming to represent, display, and manipulate data as objects while being cognizant of associated risk/safety for safe operation of computing and information systems

- Optional Textbook Readings:
  - [IJPDS] Chapter 12.10 - 12.11 The File Class, File Input and Output
  - [IJPDS] Chapter 14 JavaFX Basics

# Coursework Feedback

- CW1 Week 6 Feedback on student submissions:
  - case matters in Java!

```java
// CW1 #6.1 SET COORDINATES
public void setCoordinates(double x, double y) {
    this.X=x;
    this.Y=y;



}
```

Check

**Syntax Error(s)**

```
__tester__.java:34: error: cannot find symbol
    this.X=x;
        ^
  symbol: variable X
__tester__.java:35: error: cannot find symbol
    this.Y=y;
        ^
  symbol: variable Y
2 errors
```

# Part 1: File I/O

- In the first part of lecture, we will learn about creating, writing to and reading from a file

# File and File Class

- Data stored in a variable will be lost when a program terminates
- We need to save the data into a file, in a directory
  - the file is then stored into a storage device
- Absolute file name – depends on the OS – for example in Windows:
  - C:\Users\erick.purwanto\Documents\NetBeansProjects\Lecture11Demo\data\text1.txt
- Relative file name – relative to the current working directory
  - data\text1.txt
- In Java, we use a `File` object to represent a file
  - `File file = new File("C:\\Users\\erick.Purwanto\\Documents\\NetBeansProjects\\Lecture11Demo\\data\\text1.txt");`

# File Object

- Since OS may vary, **do not** use absolute file name when creating `File` objects
- Use relative file name in Java (with forward slash) :
  `File file = new File("data/text1.txt");`
- Note that creating a File object **does not** actually create a file on the computer!

# File Methods

- `File` class contains method to obtain the property of a file/directory

```
File file = new File("data/text1.txt");
file.exists();
file.isFile();
file.isDirectory();
file.isHidden();
file.length();
file.canRead();
file.canWrite();
file.getAbsolutePath();
```

in bytes

  - and to rename and delete a file/directory

    `delete(), renameTo(File), mkdir()`

however, to read/write into the file we need another class

# Creating a new file using PrintWriter

- `PrintWriter` class is used to create a file and write data into a text file

```
File newFile = new File("data/text2.txt");
if (newFile.exists()) {
    System.out.println("File already exists!");
    System.exit(0);
}

try {
    PrintWriter output = new PrintWriter(newFile);
} catch (IOException ioe) {
    System.out.println(ioe.getMessage());
}
```

checking for existence first so that not overwritting

if so, print message and exit the program

otherwise, the new file will be created

Java forces us to handle this exception

# Writing data into an existing file using PrintWriter

- `PrintWriter` class is used to create a file and write data into a text file

```
File file = new File("data/text1.txt");
try {
    PrintWriter output = new PrintWriter(file);
    output.println("Tanjiro");
    output.println("Zenitsu");
    output.println("Inosuke");
    output.close();
} catch (IOException ioe) {
    System.out.println(ioe.getMessage());
}
```

assuming file exists

then write similar to `System.out.println`

`close()` method must be used to write the data and close the file

# In-Class Quiz 10.1

- What happens if we run the program again?

```
File file = new File("data/text1.txt");
try {
    PrintWriter output = new PrintWriter(file);
    output.println("Tanjiro");
    output.println("Zenitsu");
    output.println("Inosuke");
    output.close();
} catch (IOException ioe) {
    System.out.println(ioe.getMessage());
}
```

  - 3 more lines added
  - no more lines added
  - there is an error message printed

# Improve efficiency using BufferedWriter

- BufferedWriter stores the strings in memory before writing into the file

```
try {
    FileWriter file = new FileWriter("data/text1.txt");
    BufferedWriter buffer = new BufferedWriter(file);
    buffer.write("Tanjiro");
    buffer.newLine();
    buffer.write("Zenitsu");
    buffer.newLine();
    buffer.flush();
    buffer.write("Inosuke");
    buffer.newLine();
    buffer.close();
} catch (IOException ioe) {
    System.out.println(ioe.getMessage());
}
```

use flush() method to force writing buffer content into file

otherwise write until buffer is full or closed

# Reading text data using Scanner

- To read from a text file, create a Scanner for the file, read line-by-line:

```
File file = new File("data/text1.txt");
try {
    Scanner input = new Scanner(file);
    while (input.hasNextLine()) {
        String name = input.nextLine();
        Swordsman swordsman = new Swordsman(name);
        System.out.println(swordsman);
    }
    input.close();
} catch (IOException ioe) {
    System.out.println(ioe.getMessage());
}
```

# Reading csv data using Scanner

- To read from a csv (comma-separated values) file, create a Scanner for the file, extract the values by first splitting them:

```
File file = new File("data/demonslayerdata.csv");
try {
    Scanner input = new Scanner(file);
    while (input.hasNextLine()) {
        String input = input.nextLine();
        String[] values = line.split(",");
        String name = values[0];
        int numDemonsKilled = Integer.parseInt(values[1]);
        Swordsman swordsman = new Swordsman(name, numDemonsKilled);
        System.out.println(swordsman);
    }
    input.close();
} catch (IOException ioe) {
    System.out.println(ioe.getMessage());
}
```

extract data from a line, line-by-line

# Improve efficiency using BufferedReader

- BufferedReader reads once from file into memory to accessed later

```
File file = new File("data/text1.txt");
try {
    FileReader fileReader = new FileReader(file);
    BufferedReader reader = new BufferedReader(fileReader);
    String line;
    while ((line = reader.readLine()) != null) {
        System.out.println(line);
    }
    reader.close();
} catch (IOException ioe) {
    System.out.println(ioe.getMessage());
}
```

access all lines line-by-line

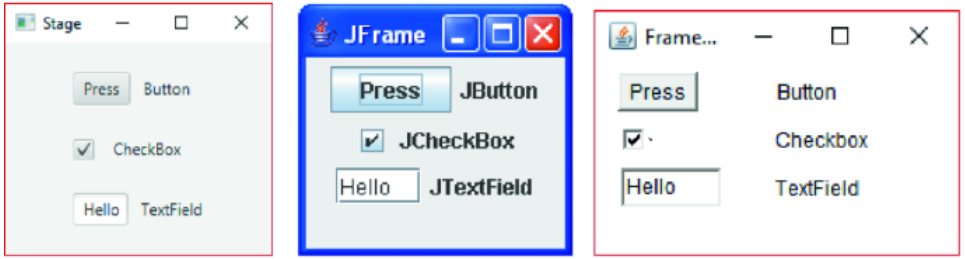
# Part 2: GUI with JavaFX

- In the second part of lecture, we will learn about GUI with JavaFX
  - how to develop **Java GUI** (Graphical User Interface) programs

- Sources:
  - Introduction to Java Programming and Data Structure, 12th Edition, Y. Daniel Liang, **Chapter 14 – Java FX Basics** for the main reading
    - please read for full codes and explanations
  - **Chapter 15-16** for self-study

# JavaFX vs Swing/AWT

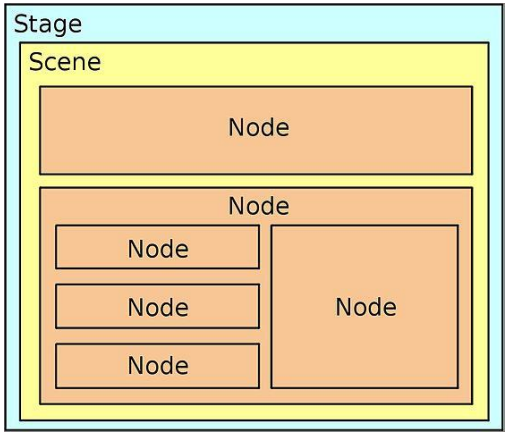
- Previously, Java GUI classes are Abstract Windows Toolkit (AWT) and Swing
  - Depreciated, no longer updated
- JavaFX is the **new** framework that **replaces** them since Java 8
  - For **CW3**, you **must** use JavaFX
  - Submissions with other platforms such as Swing/AWT are **not** accepted




JavaFX Swing AWT

# JavaFX Advantages

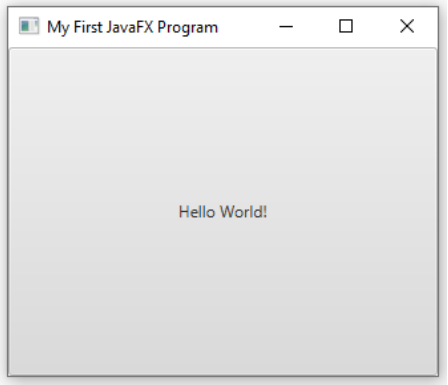
- **Simple** and excellent application of **OOP** with different layers:
  - **Stage**: top level container to be displayed
  - **Scene**: Scene objects are placed in a Stage object
  - **Node**: Node objects are placed in a Scene object
    - Node objects are visual components such as buttons, images, panes, and shapes
    - Node can contain other nodes

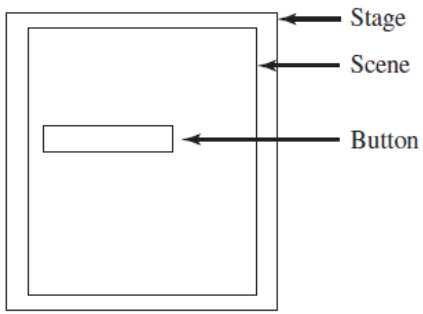


# Hello GUI World!

- Our first JavaFX Program:

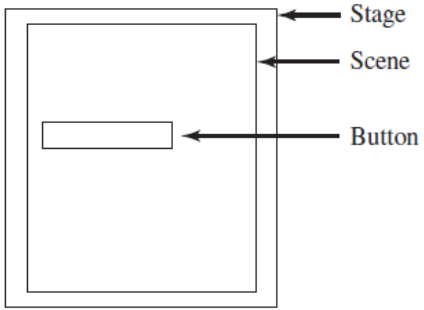
```java
import javafx.application.Application;
import javafx.stage.Stage;
import javafx.scene.Scene;
import javafx.scene.control.Button;

public class JavaFXTesting extends Application {

    @Override
    public void start(Stage primaryStage) {
        Button helloButton = new Button("Hello World!");
        Scene scene = new Scene(helloButton, 200, 250);
        primaryStage.setTitle("My First JavaFX Program");
        primaryStage.setScene(scene);
        primaryStage.show();
    }

    public static void main(String[] args) {
        launch(args);
    }

}
```



# Stage and Scene

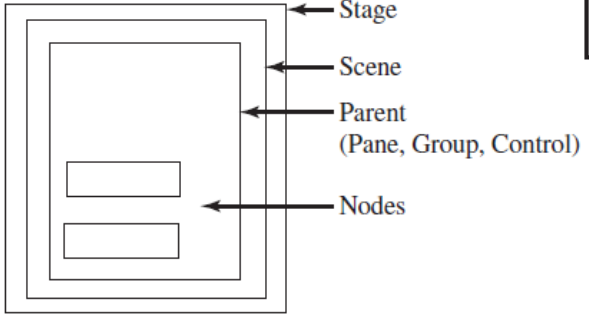
- The main class overrides **start** method defined in **Application**
  - start method **places** node objects such as a **Button** object in a **Scene** object
  - and **displays** Scene objects in a **Stage** object

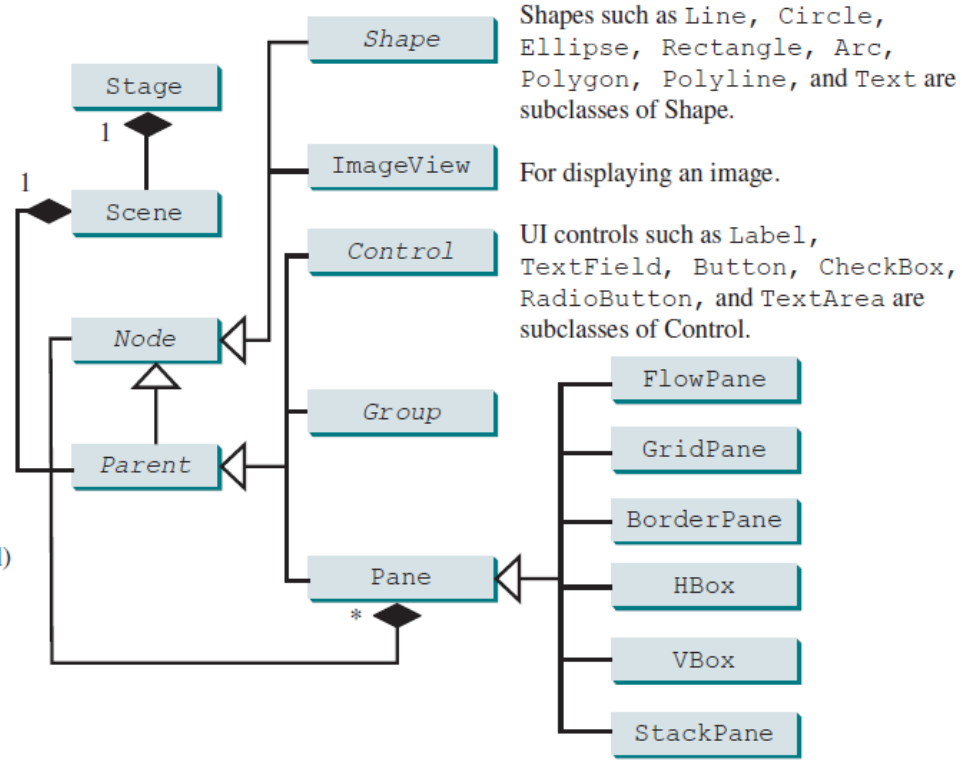



- Scene objects are created by **Scene(node, width, height)**
  - the constructor specifies width, height and places **node** in the scene
- A Stage object is a window
  - a Stage object called primary stage is automatically created by JVM when the application is launched

# Panes and Group

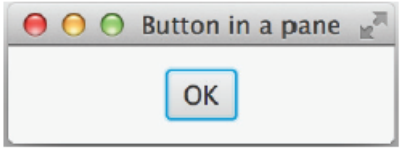
- Place nodes inside a pane to set the location and the size in a scene
- Group a collection of nodes into a Group object



# Button in a Pane

- A button is placed in the center of the pane



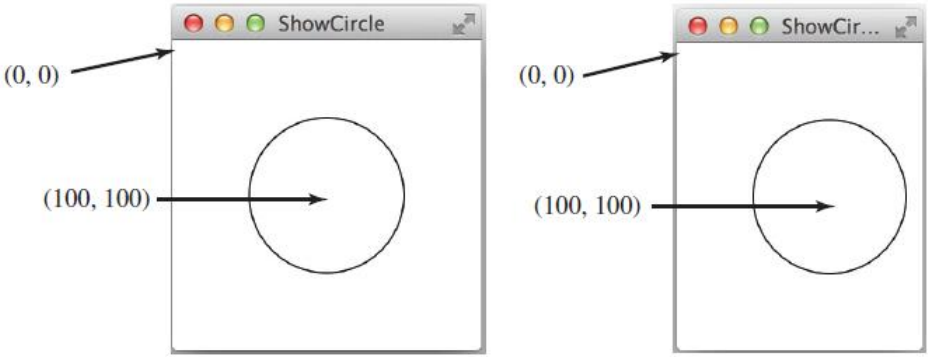

```java
import javafx.application.Application;
import javafx.scene.Scene;
import javafx.scene.control.Button;
import javafx.stage.Stage;
import javafx.scene.layout.StackPane;

public class ButtonInPane extends Application {
  @Override // Override the start method in the Application class
  public void start(Stage primaryStage) {
    // Create a scene and place a button in the scene
    StackPane pane = new StackPane();
    pane.getChildren().add(new Button("OK"));
    Scene scene = new Scene(pane, 200, 50);
    primaryStage.setTitle("Button in a pane"); // Set the stage title
    primaryStage.setScene(scene); // Place the scene in the stage
    primaryStage.show(); // Display the stage
  }
}
```

```java
StackPane pane = new StackPane(new Button("OK"));
```

# Circle

- Circle is displayed in the center of scene, but not after window is resized

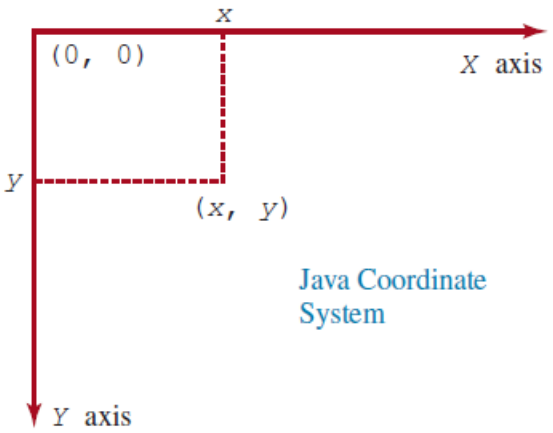
```
// Create a circle and set its properties
Circle circle = new Circle();
circle.setCenterX(100);
circle.setCenterY(100);
circle.setRadius(50);
circle.setStroke(Color.BLACK);
circle.setFill(Color.WHITE);

// Create a pane to hold the circle
Pane pane = new Pane();
pane.getChildren().add(circle);

// Create a scene and place it in the stage
Scene scene = new Scene(pane, 200, 200);
primaryStage.setTitle("ShowCircle"); // Set the stage title
primaryStage.setScene(scene); // Place the scene in the stage
primaryStage.show(); // Display the stage
```

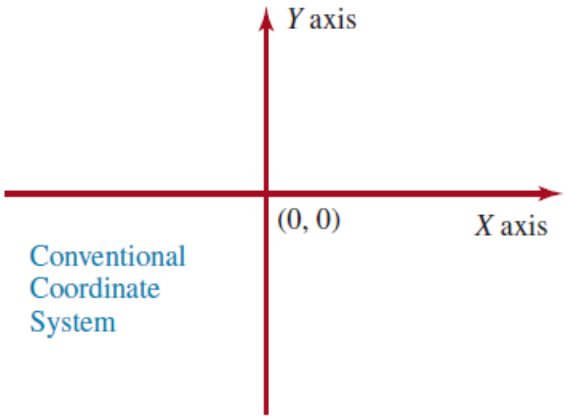



*please check **main reading** for complete code, we only display code/partial code inside **start** method*

# Java Coordinate System

- Java coordinate system is measured in pixels, with (0, 0) at its upper-left corner



# Property Binding to Center Circle on Resizing

- Property binding enables *a target object* to be bound to *a source object*
  - to automatically change centerX, centerY, **bind** them with width/2, height/2

```
// Create a pane to hold the circle
Pane pane = new Pane();

// Create a circle and set its properties
Circle circle = new Circle();
circle.centerXProperty().bind(pane.widthProperty().divide(2));
circle.centerYProperty().bind(pane.heightProperty().divide(2));
circle.setRadius(50);
circle.setStroke(Color.BLACK);
circle.setFill(Color.WHITE);
pane.getChildren().add(circle); // Add circle to the pane

// Create a scene and place it in the stage
Scene scene = new Scene(pane, 200, 200);
primaryStage.setTitle("ShowCircleCentered"); // Set the stage title
primaryStage.setScene(scene); // Place the scene in the stage
primaryStage.show(); // Display the stage
```

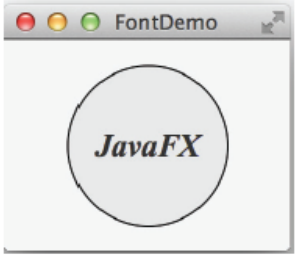
# Color

- **Color** class is used to create colors
- Use **RGBA** (red, green, blue, alpha/opacity) model
  - values are from **0.0** (darkest) to **1.0** (lightest)
  - example: **Color** color = new **Color**(0.25, 0.14, 0.333, 0.51);
- Alternatively, use constants for standard colors
  - BEIGE, BLACK, BLUE, BROWN, CYAN, DARKGRAY, GOLD, GRAY, GREEN, LIGHTGRAY, MAGENTA, NAVY, ORANGE, PINK, RED, SILVER, WHITE, and YELLOW
  - example: circle.setFill(**Color.RED**);

# Font, Label and StackPane

- **Font** is defined by its name, weight, posture, and size; and is set on a **Label** object

```
// Create a pane to hold the circle
Pane pane = new StackPane();

// Create a circle and set its properties
Circle circle = new Circle();
circle.setRadius(50);
circle.setStroke(Color.BLACK);
circle.setFill(new Color(0.5, 0.5, 0.5, 0.1));
pane.getChildren().add(circle); // Add circle to the pane

// Create a label and set its properties
Label label = new Label("JavaFX");
label.setFont(Font.font("Times New Roman",
  FontWeight.BOLD, FontPosture.ITALIC, 20));
pane.getChildren().add(label);

// Create a scene and place it in the stage
Scene scene = new Scene(pane);
primaryStage.setTitle("FontDemo"); // Set the stage title
primaryStage.setScene(scene); // Place the scene in the stage
primaryStage.show(); // Display the stage
```




- A **StackPane** object is used to contain a circle and a label

# Image and ImageView

- **Image** class loads an image; **ImageView** is a node for displaying image

```
// Create a pane to hold the image views
Pane pane = new HBox(10);
pane.setPadding(new Insets(5, 5, 5, 5));
Image image = new Image("image/us.gif");
pane.getChildren().add(new ImageView(image));

ImageView imageView2 = new ImageView(image);
imageView2.setFitHeight(100);
imageView2.setFitWidth(100);
pane.getChildren().add(imageView2);

ImageView imageView3 = new ImageView(image);
imageView3.setRotate(90);
pane.getChildren().add(imageView3);

// Create a scene and place it in the stage
Scene scene = new Scene(pane);
primaryStage.setTitle("ShowImage"); // Set the stage title
primaryStage.setScene(scene); // Place the scene in the stage
primaryStage.show(); // Display the stage
```

- An **HBox** is a pane that places all nodes horizontally in one row

# Layout Panes

- JavaFX provides many types of panes for organizing nodes

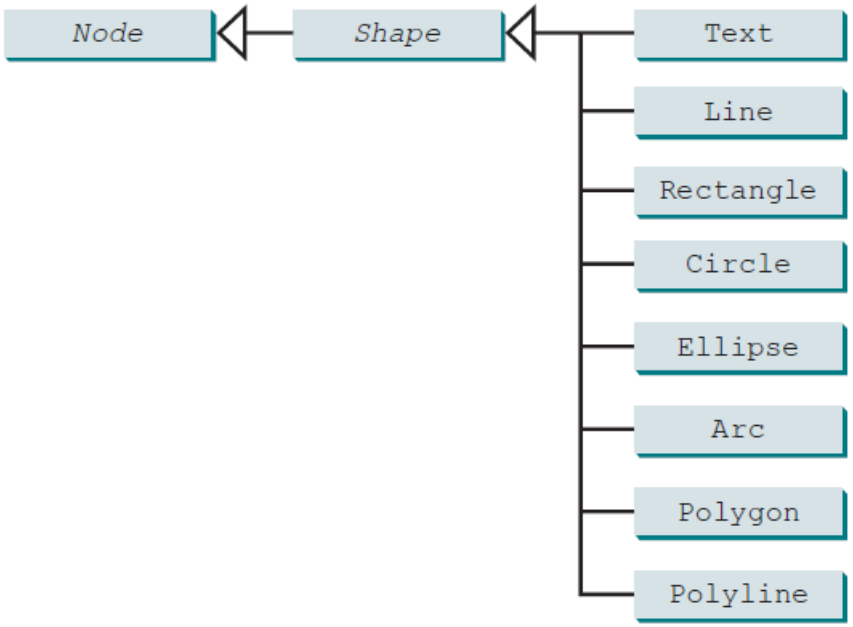
| *Class* | *Description* |
|---|---|
| `Pane` | Base class for layout panes. It contains the `getChildren()` method for returning a list of nodes in the pane. |
| `StackPane` | Places the nodes on top of each other in the center of the pane. |
| `FlowPane` | Places the nodes row-by-row horizontally or column-by-column vertically. |
| `GridPane` | Places the nodes in the cells in a two-dimensional grid. |
| `BorderPane` | Places the nodes in the top, right, bottom, left, and center regions. |
| `HBox` | Places the nodes in a single row. |
| `VBox` | Places the nodes in a single column. |

*please check **main reading** for the details*

# Shapes

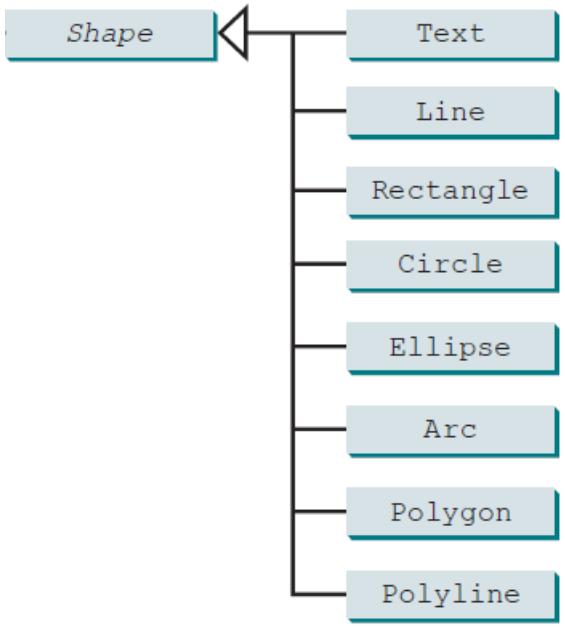
- Shapes are nodes used for drawing texts, lines, texts, lines, circles, rectangles, ellipses, arcs, polygons, and polylines;  with properties:

  - **fill**: a color fills the interior of a shape
  - **stroke**: a color used to draw the outline of a shape
  - **strokeWidth**: width of outline of a shape



# In-Class Quiz 10.2

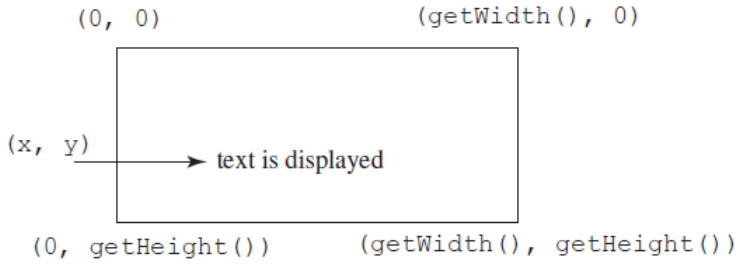
- If we want to draw **a triangle**, which JavaFX Shape should we use?
  - Line
  - Rectangle
  - Arc
  - Polygon
  - Polyline



# Text

- **Text** displays a string at a starting point (x, y), placed in a pane

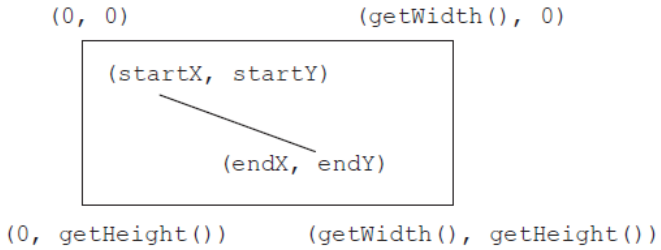
```
// Create a pane to hold the texts
Pane pane = new Pane();
pane.setPadding(new Insets(5, 5, 5, 5));
Text text1 = new Text(20, 20, "Programming is fun");
text1.setFont(Font.font("Courier", FontWeight.BOLD,
  FontPosture.ITALIC, 15));
pane.getChildren().add(text1);

Text text2 = new Text(60, 60, "Programming is fun\nDisplay text");
pane.getChildren().add(text2);

Text text3 = new Text(10, 100, "Programming is fun\nDisplay text");
text3.setFill(Color.RED);
text3.setUnderline(true);
text3.setStrikethrough(true);
pane.getChildren().add(text3);

// Create a scene and place it in the stage
Scene scene = new Scene(pane);
primaryStage.setTitle("ShowText"); // Set the stage title
primaryStage.setScene(scene); // Place the scene in the stage
primaryStage.show(); // Display the stage
```

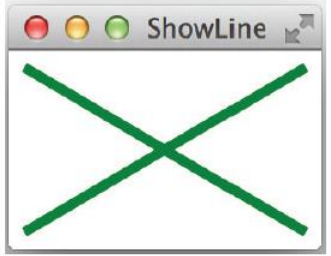


# Line

- **Line** connects two points with four parameters **startX**, **startY**, **endX**, and **endY**

```java
public class ShowLine extends Application {
  @Override // Override the start method in the Application class
  public void start(Stage primaryStage) {
    // Create a scene and place it in the stage
    Scene scene = new Scene(new LinePane(), 200, 200);
    primaryStage.setTitle("ShowLine"); // Set the stage title
    primaryStage.setScene(scene); // Place the scene in the stage
    primaryStage.show(); // Display the stage
  }
}
class LinePane extends Pane {
  public LinePane() {
    Line line1 = new Line(10, 10, 10, 10);
    line1.endXProperty().bind(widthProperty().subtract(10));
    line1.endYProperty().bind(heightProperty().subtract(10));
    line1.setStrokeWidth(5);
    line1.setStroke(Color.GREEN);
    getChildren().add(line1);

    Line line2 = new Line(10, 10, 10, 10);
    line2.startXProperty().bind(widthProperty().subtract(10));
    line2.endYProperty().bind(heightProperty().subtract(10));
    line2.setStrokeWidth(5);
    line2.setStroke(Color.GREEN);
    getChildren().add(line2);
  }
}
```



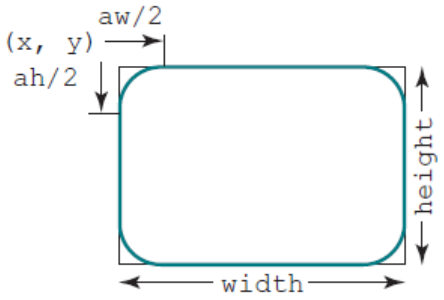



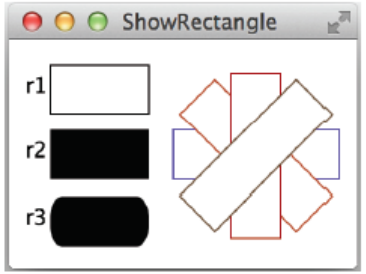

- A custom pane class, **LinePane**, creates two lines and binds the starting, ending points with width, height of pane so two points are changed when pane is resized

# Rectangle

- **Rectangle** is defined by **x**, **y**, **width**, **height**, **arcWidth**, and **arcHeight**

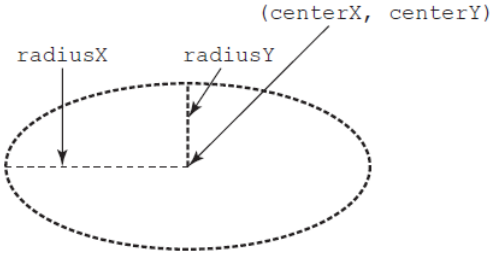
```
// Create rectangles
Rectangle r1 = new Rectangle(25, 10, 60, 30);
r1.setStroke(Color.BLACK);
r1.setFill(Color.WHITE);
Rectangle r2 = new Rectangle(25, 50, 60, 30);
Rectangle r3 = new Rectangle(25, 90, 60, 30);
r3.setArcWidth(15);
r3.setArcHeight(25);
// Create a group and add nodes to the group
Group group = new Group();
group.getChildren().addAll(new Text(10, 27, "r1"), r1,
  new Text(10, 67, "r2"), r2, new Text(10, 107, "r3"), r3);

for (int i = 0; i < 4; i++) {
  Rectangle r = new Rectangle(100, 50, 100, 30);
  r.setRotate(i * 360 / 8);
  r.setStroke(Color.color(Math.random(), Math.random(),
    Math.random()));
  r.setFill(Color.WHITE);
  group.getChildren().add(r);
}
// Create a scene and place it in the stage
Scene scene = new Scene(new BorderPane(group),250, 150);
```



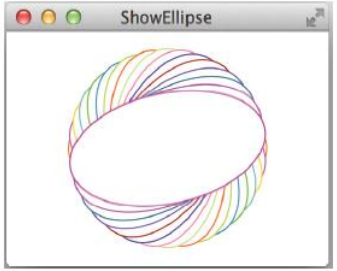



# Ellipse

- **Ellipse** is defined by **centerX**, **centerY**, **radiusX**, and **radiusY**




```java
// Create a scene and place it in the stage
Scene scene = new Scene(new MyEllipse(), 300, 200);

class MyEllipse extends Pane {
  private void paint() {
    getChildren().clear();
    for (int i = 0; i < 16; i++) {
      // Create an ellipse and add it to pane
      Ellipse e1 = new Ellipse(getWidth() / 2, getHeight() / 2,
        getWidth() / 2 - 50, getHeight() / 2 - 50);
      e1.setStroke(Color.color(Math.random(), Math.random(),
        Math.random()));
      e1.setFill(Color.WHITE);
      e1.setRotate(i * 180 / 16);
      getChildren().add(e1);
    }
  }

  @Override
  public void setWidth(double width) {
    super.setWidth(width);
    paint();
  }

  @Override
  public void setHeight(double height) {
    super.setHeight(height);
    paint();
  }
```

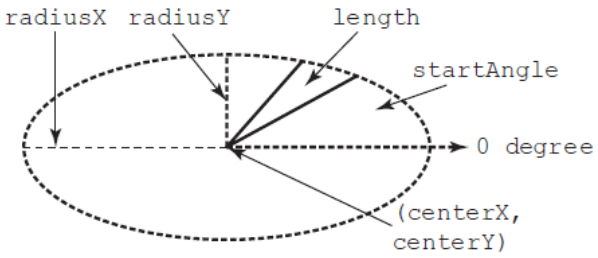


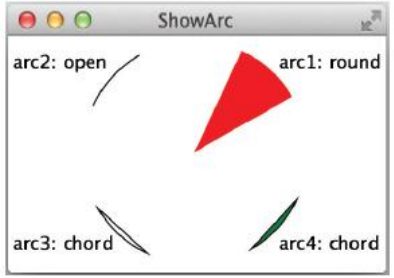

- By defining **MyEllipse** class, we can reuse it, and since it can be resized since it extends Pane and overrides setWidth and setHeight methods

# Arc

- **Arc**, part of an ellipse, defined by **centerX**, **centerY**, **radiusX**, **radiusY**, **startAngle**, **length**, and **an arc type** (ArcType.OPEN, ArcType.CHORD, or ArcType.ROUND)

```
Arc arc1 = new Arc(150, 100, 80, 80, 30, 35); // Create an arc
arc1.setFill(Color.RED); // Set fill color
arc1.setType(ArcType.ROUND); // Set arc type

Arc arc2 = new Arc(150, 100, 80, 80, 30 + 90, 35);
arc2.setFill(Color.WHITE);
arc2.setType(ArcType.OPEN);
arc2.setStroke(Color.BLACK);

Arc arc3 = new Arc(150, 100, 80, 80, 30 + 180, 35);
arc3.setFill(Color.WHITE);
arc3.setType(ArcType.CHORD);
arc3.setStroke(Color.BLACK);

Arc arc4 = new Arc(150, 100, 80, 80, 30 + 270, 35);
arc4.setFill(Color.GREEN);
arc4.setType(ArcType.CHORD);
arc4.setStroke(Color.BLACK);
// Create a group and add nodes to the group
Group group = new Group();
group.getChildren().addAll(new Text(210, 40, "arc1: round"),
  arc1, new Text(20, 40, "arc2: open"), arc2,
  new Text(20, 170, "arc3: chord"), arc3,
  new Text(210, 170, "arc4: chord"), arc4);
// Create a scene and place it in the stage
Scene scene = new Scene(new BorderPane(group), 300, 200);
```

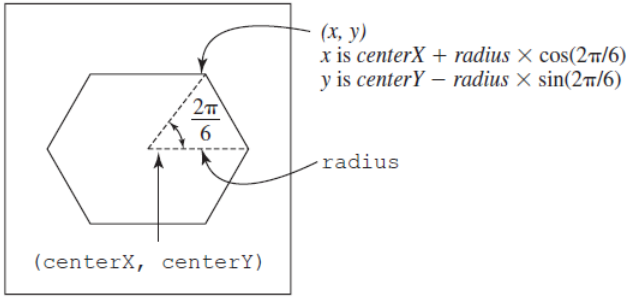


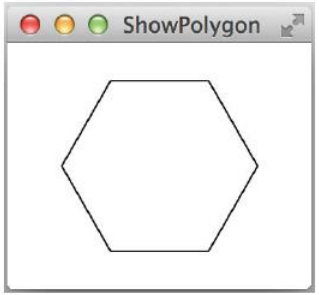



# Polygon and Polyline

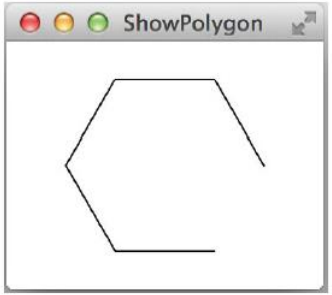
- **Polygon** connects a sequence of points

```
class MyPolygon extends Pane {
  private void paint() {
    // Create a polygon and place polygon to pane
    Polygon polygon = new Polygon();
    polygon.setFill(Color.WHITE);
    polygon.setStroke(Color.BLACK);
    ObservableList<Double> list = polygon.getPoints();

    double centerX = getWidth() / 2, centerY = getHeight() / 2;
    double radius = Math.min(getWidth(), getHeight()) * 0.4;

    // Add points to the polygon list
    for (int i = 0; i < 6; i++) {
      list.add(centerX + radius * Math.cos(2 * i * Math.PI / 6));
      list.add(centerY - radius * Math.sin(2 * i * Math.PI / 6));
    }

    getChildren().clear();
    getChildren().add(polygon);
  }

  @Override
  public void setWidth(double width) {
    super.setWidth(width);
    paint();
  }
  @Override
  public void setHeight(double height) {
    super.setHeight(height);
    paint();
  }
}
```




- If replace Polygon with **Polyline**, similar display, except the starting and ending points are not connected



# Thank you for your attention !

- In this lecture,  you have learned:
  - how to create, read and write files
  - how develop rich GUI applications with JavaFX, using a stage, scenes and nodes
  - how to draw shapes such as circles, lines, rectangles, ellipses, arcs, polygons, and polylines

- Please continue to Lab 10 to complete Lab Tasks
  - and to practice Pair Programming and Drawing with JavaFX!